

One Pricer a Day

Neel Dev Sen

July 2026

Contents

1	Introduction	3
I	European Options	4
2	Closed-Form Solutions	5
2.1	Day 1: Black-Scholes Equation	5
2.1.1	Risk-Neutral Measure	5
2.1.2	Derivation of the Black-Scholes PDE	5
2.1.3	Closed-Form Solution for European Call Options	8
2.1.4	Closed-Form Solution for European Put Options	10
2.2	Day 2: Black-76 PDE	11
2.2.1	Derivation of the Black-76 PDE	11
2.2.2	Black-76 Formula for Call Options	13
2.2.3	Black-76 Formula for Put Options	14
2.3	Day 3: Merton Dividend-Adjusted Black-Scholes	15

1 Introduction

This is a project where I develop derivative pricers using **C++** and occasionally **Python**. My goal is to do at least one a day and you can access all of my source files and header files here: **GitHub Repo**

Part I

European Options

2 Closed-Form Solutions

2.1 Day 1: Black-Scholes Equation

The first option that I am going to be pricing is the European option using the closed-form Black-Scholes equation.

In **GitHub** the full **C++** script is on this link: **source file**

The **header file** (first listing) is on this link: **header file**

2.1.1 Risk-Neutral Measure

The Black-Scholes equation takes the assumption of the **risk-neutral pricing measure**

$$dS_t = rS_t dt + \sigma S_t dW_t \quad (2.1)$$

[1]

where S_t is the price of the asset at a given time t , dS_t is the infinitesimal change in the asset's price, r is the risk-free rate of interest, dt is the infinitesimal change in time, σ is the implied volatility and dW_t is the infinitesimal change in the **Wiener F function**. The **Wiener function** is defined with the property:

$$W_T - W_t \sim \mathcal{N}(0, \sqrt{T - t}) \quad (2.2)$$

where $\mathcal{N}(\mu, \sigma)$ is the **normal distribution** with mean μ and standard-deviation σ . [2]

2.1.2 Derivation of the Black-Scholes PDE

Ito's Lemma states that:

$$df(t, S_t) = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S_t} dS_t + \frac{1}{2} \frac{\partial^2 f}{\partial S_t^2} (dS_t)^2 \quad (2.3)$$

Substituting equation 2.1 for dS_t we get:

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S_t} (rS_t dt + \sigma S_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial S_t^2} (rS_t dt + \sigma S_t dW_t)^2 \\ &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S_t} (rS_t dt + \sigma S_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial S_t^2} (r^2 S_t^2 (dt)^2 + 2r\sigma S_t^2 (dt)(dW_t) + \sigma^2 S_t^2 (dW_t)^2) \end{aligned} \quad (2.4)$$

The **Ito's multiplication rules** state that:

$$(dt)^2 = 0 \quad (2.5)$$

$$(dt)(dW_t) = 0 \quad (2.6)$$

$$(dW_t)^2 = dt \quad (2.7)$$

Substituting equations 2.5, 2.6, and 2.7 into equation 2.4 we get:

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S_t} (rS_t dt + \sigma S_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial S_t^2} (0 + 0 + \sigma^2 S_t^2 (dt)) \\ &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S_t} (rS_t dt + \sigma S_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial S_t^2} \sigma^2 S_t^2 \\ &= \left(\frac{\partial f}{\partial t} + rS_t \frac{\partial f}{\partial S_t} + \frac{1}{2} \sigma^2 S_t^2 \frac{\partial^2 f}{\partial S_t^2} \right) dt + \sigma S_t \frac{\partial f}{\partial S_t} dW_t \end{aligned} \quad (2.8)$$

Define

$$V(t, S) \equiv f(t, S_t) \quad (2.9)$$

Using the result from equation 2.8

$$dV(t, S) = \left(\frac{\partial V}{\partial t} + rS \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt + \sigma S \frac{\partial V}{\partial S} dW_t \quad (2.10)$$

Now we will define a delta-hedged portfolio Π and $\Delta = \frac{\partial V}{\partial S}$ that satisfies:

$$\Pi = V - \Delta S \quad (2.11)$$

For an infinitesimal time step this becomes:

$$d\Pi = dV - \Delta dS \quad (2.12)$$

Substituting equation 2.10 for dV and equation 2.1 for dS we get.

$$\begin{aligned} d\Pi &= \left(\frac{\partial V}{\partial t} + rS \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt + \sigma S \frac{\partial V}{\partial S} dW_t - \Delta (rS dt + \sigma S dW_t) \\ &= \left(\frac{\partial V}{\partial t} + rS \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - \Delta rS \right) dt + (\sigma S \frac{\partial V}{\partial S} - \Delta \sigma S) dW_t \end{aligned} \quad (2.13)$$

Since $\Delta = \frac{\partial V}{\partial S}$, equation 2.13 becomes:

$$\begin{aligned}
d\Pi &= \left(\frac{\partial V}{\partial t} + rS \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rS \frac{\partial V}{\partial S} \right) dt + \left(\sigma S \frac{\partial V}{\partial S} - \sigma S \frac{\partial V}{\partial S} \right) dW_t \\
&= \left(\frac{\partial V}{\partial t} + rS \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rS \frac{\partial V}{\partial S} \right) dt \\
&= \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt
\end{aligned} \tag{2.14}$$

The **arbitrage condition** in a **risk-free portfolio** is:

$$d\Pi = r\Pi dt \tag{2.15}$$

Substituting this into equation 2.14 we get:

$$\begin{aligned}
r\Pi dt &= \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt \\
r\Pi &= \frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}
\end{aligned} \tag{2.16}$$

Substituting equation 2.11 into equation 2.16 we get:

$$\begin{aligned}
r(V - \Delta S) &= \frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \\
r\left(V - \frac{\partial V}{\partial S} S\right) &= \frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \\
rV - rS \frac{\partial V}{\partial S} &= \frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}
\end{aligned} \tag{2.17}$$

This gives us the final partial differential equation which is also the **Black-Scholes equation**:

$$\boxed{\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0} \tag{2.18}$$

In this equation V is the option's price, t is the current time. $\frac{\partial V}{\partial t}$ is the rate of change of the option's price to time, $\frac{\partial V}{\partial S}$ is the rate of change of the option's price to the underlying stock price and $\frac{\partial^2 V}{\partial S^2}$ is a concavity factor on how the option price changes with the stock price. The Black-Scholes can be solved for a call option and a put option. [3].

2.1.3 Closed-Form Solution for European Call Options

The closed-form solution for the Black-Scholes equation where the option is a call option is:

$$\begin{aligned} C(t, S) &= S\Phi(d_1) - Ke^{-r(T-t)}\Phi(d_2) \\ d_1 &= \frac{\ln(\frac{S}{K}) + (r + \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}} \\ d_2 &= d_1 - \sigma\sqrt{T-t} \end{aligned} \tag{2.19}$$

In this equation $T-t$ represents the time to maturity and K represents the strike price. [3].

$\Phi(z)$ is the standard normal cumulative distribution function which can be represented with the error function $\text{erf}(z)$:

$$\Phi(z) = \frac{1}{2}(1 + \text{erf}(\frac{z}{\sqrt{2}})) \tag{2.20}$$

In C++ this can be implemented as:

```
1 #include <iostream>
2 #include <cmath>
3 #include <type_traits>
4
5 template <typename T>
6 auto normalCDF(T x) -> std::common_type_t <double , T>
7 {
8     using commonType = std::common_type_t<double , T>;
9     return static_cast<commonType>(0.5) * (static_cast<
10         commonType>(1) + std::erf(x / std::sqrt(2.0)));
11 }
```

The function we are creating is called **normalCDF**. This listing uses the **cmath** header file in order to import **std::erf**. It also uses a function template with a type placeholder **T** so that we can use this function for any type given. In order to ensure that this function has at least a type **double** we use the header file **type traits** which allows us to specify that we want this function to be at least double in line 6 trailing return type. In line 8 we ensure that the **commonType** alias used amongst all the literals and variables in this function are also at least of type **double**. This function returns the result shown in equation 2.20 with a type of at least **double**.

Now to implement the function shown in equation 2.19 in C++ we use:

```

1 #include "functions.hpp" // header file for normalCDF
2
3 template <typename SType, typename KType, typename rType, typename
   sigmaType, typename tType>
4 auto blackScholesCall(SType S, KType K, rType r, sigmaType sigma,
   tType t) -> std::common_type_t<double, SType, KType, rType,
   sigmaType, tType>
5 {
6     using commonType = std::common_type_t<double, SType, KType,
   rType, sigmaType, tType>;
7     auto S_new {static_cast<commonType>(S)};
8     auto K_new {static_cast<commonType>(K)};
9     auto r_new {static_cast<commonType>(r)};
10    auto sigma_new {static_cast<commonType>(sigma)};
11    auto t_new {static_cast<commonType>(t)};
12    auto sqrt_t_new {std::sqrt(t_new)};
13
14    auto d1 {(std::log(S_new / K_new) + (r_new + (static_cast<
   commonType>(0.5) * sigma_new * sigma_new) * t_new) / (
   sigma_new * sqrt_t_new))};
15    auto d2 {d1 - sigma_new * sqrt_t_new};
16
17    auto C {S_new * normalCDF(d1) - K_new * std::exp(-r_new *
   t_new) * normalCDF(d2)};
18    return C;
19 }

```

The function in this listing is called **blackScholesCall**. This listing again uses the same header files and the same principles as the one before and converts every thing into a **commonType** of at least type **double**. The formula for d_1 is shown in line 12 and the formula for d_2 is shown in line 13. Finally the result in equation 2.19 is shown in line 16 using the previously defined function **normalCDF**. The function returns the value of the **call option** with at least type **double**.

2.1.4 Closed-Form Solution for European Put Options

The closed-form solution for the Black-Scholes equation for a put option is:

$$P(t, S) = Ke^{-r(T-t)}\Phi(-d_2) - S\Phi(-d_1) \quad (2.21)$$

using the same definitions of d_1 and d_2 in 2.19 [3]

The implementation of this in C++ is:

```
1 #include "functions.hpp" // header file for normalCDF
2
3 template <typename SType, typename KType, typename rType, typename
4   sigmaType, typename tType>
5 auto blackScholesPut(SType S, KType K, rType r, sigmaType sigma,
6   tType t) -> std::common_type_t<double, SType, KType, rType,
7   sigmaType, tType>
8 {
9     using commonType = std::common_type_t<double, SType, KType,
10     rType, sigmaType, tType>;
11     auto S_new {static_cast<commonType>(S)};
12     auto K_new {static_cast<commonType>(K)};
13     auto r_new {static_cast<commonType>(r)};
14     auto sigma_new {static_cast<commonType>(sigma)};
15     auto t_new {static_cast<commonType>(t)};
16     auto sqrt_t_new {std::sqrt(t_new)};
17
18     auto d1 {(std::log(S_new / K_new) + (r_new + (static_cast<
19     commonType>(0.5) * sigma_new * sigma_new) * sqrt_t_new) /
20     (sigma_new * sqrt_t_new)};
21     auto d2 {d1 - sigma_new * std::sqrt(t_new)};
22
23     auto P {K_new * std::exp(-r_new * t_new) * normalCDF(-d2) -
24     S_new * normalCDF(-d1)};
25     return P;
26 }
```

The function in this listing is called **blackScholesPut**. This listing is almost identical to the one for **blackScholesCall**. The formula for d_1 is again shown in line 12 and the formula for d_2 is shown in line 13. Finally the result in equation 2.21 is computed in line 16 (stored as **P**) using the function **normalCDF**. The function returns the value of the **put option** with at least type **double**.

2.2 Day 2: Black-76 PDE

The Black-76 model is just an adaption of the **Black-Scholes equation** except it uses the price of futures and bonds to price options.

In **GitHub** the source file can be found here: **source file**

For the listings in this section I will continue using the header file **functions.hpp** that I defined on day 1. I will continue to use the **normalCDF** function from there.

2.2.1 Derivation of the Black-76 PDE

The modelling assumptions are that the futures of forward price process is modelled by **geometric Brownian motion** shown by the equation:

$$dF_t = \sigma F_t dW_t \quad (2.22)$$

where F_t is the price of a future or a forward at a given time, dF_t is the infinitesimal change of the price of a future of a forward, σ is implied volatility and dW_t is the infinitesimal change in the **Wiener function**.^[4]

Substituting equation 2.22 into **Ito's Lemma 2.3**

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial F_t} (\sigma F_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial F_t^2} (\sigma F_t dW_t)^2 \\ &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial F_t} (\sigma F_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial F_t^2} (\sigma^2 F_t^2 (dW_t)^2) \end{aligned} \quad (2.23)$$

Substituting equations 2.7 into equation 2.23 we get:

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial F_t} (\sigma F_t) dW_t + \frac{1}{2} \frac{\partial^2 f}{\partial F_t^2} (\sigma^2 F_t^2) dt \\ &= \left(\frac{\partial f}{\partial t} + \frac{1}{2} \sigma^2 F_t^2 \frac{\partial^2 f}{\partial F_t^2} \right) dt + \sigma F_t \frac{\partial f}{\partial F_t} dW_t \end{aligned} \quad (2.24)$$

Define

$$V(t, S) \equiv f(t, S_t) \quad (2.25)$$

Using the result from equation 2.24

$$dV(t, S) = \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S_t^2 \frac{\partial^2 V}{\partial F^2} \right) dt + \sigma F \frac{\partial V}{\partial F} dW_t \quad (2.26)$$

The thing that is different about the **delta-hedged portfolio** is that at the time the option is being exchanged, the cost of the **future** is 0.

$$\Pi = V \quad (2.27)$$

For an infinitesimal time step, the value of the future increases by dF which leads to

$$d\Pi = dV - \Delta dF \quad (2.28)$$

where $\Delta = \frac{\partial V}{\partial F}$. Substituting equation 2.26 for dV and equation 2.22 for dF we get.

$$\begin{aligned} d\Pi &= \left(\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2}\right)dt + \sigma F \frac{\partial V}{\partial F} dW_t - \Delta(\sigma F dW_t) \\ &= \left(\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2}\right)dt + (\sigma F \frac{\partial V}{\partial F} - \Delta \sigma F) dW_t \end{aligned} \quad (2.29)$$

Since $\Delta = \frac{\partial V}{\partial F}$, equation 2.29 becomes:

$$\begin{aligned} d\Pi &= \left(\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2}\right)dt + (\sigma F \frac{\partial V}{\partial F} - \sigma F \frac{\partial V}{\partial F}) dW_t \\ &= \left(\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2}\right)dt \end{aligned} \quad (2.30)$$

The **arbitrage condition** in a **risk-free portfolio** is:

$$d\Pi = r\Pi dt \quad (2.31)$$

Substituting this into equation 2.30 we get:

$$\begin{aligned} r\Pi dt &= \left(\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2}\right)dt \\ r\Pi &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \end{aligned} \quad (2.32)$$

Substituting equation 2.27 into equation 2.32 we get:

$$rV = \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \quad (2.33)$$

Which finally leads to the equation:

$$\boxed{\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} - rV = 0} \quad (2.34)$$

Which is also known as the **Black-76 Equation**

2.2.2 Black-76 Formula for Call Options

The Black-76 Formula for Call options is:

$$\begin{aligned} C(t, F) &= e^{-r(T-t)}(F\Phi(d_1) - K\Phi(d_2)) \\ d_1 &= \frac{\ln(\frac{F}{K}) + \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}} \\ d_2 &= d_1 - \sigma\sqrt{T-t} \end{aligned} \tag{2.35}$$

[4] The implementation in C++ is:

```
1 #include "functions.hpp" // header file for normalCDF
2
3 template <typename FType, typename KType, typename rType, typename
4   sigmaType, typename tType>
5 auto black76Call(FType F, KType K, rType r, sigmaType sigma, tType t
6   ) -> std::common_type_t<double, FType, KType, rType, sigmaType,
7   tType>
8 {
9     using commonType = std::common_type_t<double, FType, KType,
10     rType, sigmaType, tType>;
11     auto F_new {static_cast<commonType>(F)};
12     auto K_new {static_cast<commonType>(K)};
13     auto r_new {static_cast<commonType>(r)};
14     auto sigma_new {static_cast<commonType>(sigma)};
15     auto t_new {static_cast<commonType>(t)};
16     auto sqrt_t_new {std::sqrt(t_new)};
17
18     auto d1 {(std::log(F_new / K_new) + (static_cast<commonType>
19     >(0.5) * sigma_new * sigma_new) * t_new ) / (sigma_new *
20     sqrt_t_new)};
21     auto d2 {d1 - sigma_new * sqrt_t_new};
22
23     auto C {(std::exp(-r_new * t_new)) * (F_new * normalCDF(d1)
24     - K_new * normalCDF(d2))};
25     return C;
26 }
```

d_1 is calculated in line 13, d_2 is calculated in line 14 and finally equation 2.35 is computed in line 16 (stored as **C**) and returned in line 17.

2.2.3 Black-76 Formula for Put Options

The Black-76 Formula for Call options is:

$$\begin{aligned} P(t, F) &= e^{-r(T-t)}(K\Phi(-d_2) - F\Phi(-d_1)) \\ d_1 &= \frac{\ln(\frac{F}{K}) + \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}} \\ d_2 &= d_1 - \sigma\sqrt{T-t} \end{aligned} \tag{2.36}$$

[4] The implementation in C++ is:

```
1 #include "functions.hpp" // header file for normalCDF
2
3 template <typename FType, typename KType, typename rType, typename
4   sigmaType, typename tType>
5 auto black76Put(FType F, KType K, rType r, sigmaType sigma, tType t)
6   -> std::common_type_t<double, FType, KType, rType, sigmaType,
7   tType>
8 {
9     using commonType = std::common_type_t<double, FType, KType,
10     rType, sigmaType, tType>;
11     auto F_new {static_cast<commonType>(F)};
12     auto K_new {static_cast<commonType>(K)};
13     auto r_new {static_cast<commonType>(r)};
14     auto sigma_new {static_cast<commonType>(sigma)};
15     auto t_new {static_cast<commonType>(t)};
16     auto sqrt_t_new {std::sqrt(t_new)};
17
18     auto d1 {(std::log(F_new / K_new) + (static_cast<commonType>
19     >(0.5) * sigma_new * sigma_new) * t_new ) / (sigma_new *
20     sqrt_t_new)};
21     auto d2 {d1 - sigma_new * sqrt_t_new};
22
23     auto P {std::exp(-r_new * t_new) * (K_new * normalCDF(-d2) -
24     (F_new * normalCDF(-d1)))};
25     return P;
26 }
```

d_1 is calculated in line 13, d_2 is calculated in line 14 and finally equation 2.36 is computed in line 16 (stored as **P**) and returned in line 17.

2.3 Day 3: Merton Dividend-Adjusted Black-Scholes

h

References

- [1] Shreve, S. (2004). *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*. Springer Science & Business Media.
- [2] Lalley, S. P. (n.d.). *Brownian motion*. <https://galton.uchicago.edu/~lalley/Courses/313/BrownianMotionCurrent.pdf>
- [3] Haugh, M. (2016). *The Black-Scholes model* (IEOR E4706: Foundations of Financial Engineering lecture notes). Columbia University. <https://www.columbia.edu/~mh2078/FoundationsFE/BlackScholes.pdf>
- [4] Black, F. (1976). The pricing of commodity contracts. *Journal of Financial Economics*, 3(1–2), 167–179. [https://doi.org/10.1016/0304-405X\(76\)90024-6](https://doi.org/10.1016/0304-405X(76)90024-6)